PES UNIVERSITY

SOFTWARE REQUIREMENTS SPECIFICATION

Problem Statement 39
Dehallucination Add-On Module for Enhancing Reliability of AI Applications

Jackfruit Phase-1 Mini Project

Team details

Team Name	Project ID selected	Team Member 1 name and SRN per PES records	Team Member 2 name and SRN per PES records	Team Member 3 name and SRN per PES records	Team Member 4 name and SRN per PES records
FANTASTIC FOUR	31	KUSHAL J PES2UG24AM205	LAXMIKANT PES2UG24AM206	GAGAN K C PES2UG24AM201	YASHWANTH G PES2UG24AM189

Revision History

Version	Date	Description	Author
1.0	1 September 2026	Initial SRS for Problem Statement 39	Project Team

Table of Contents

1. Introduction
    1.1 Purpose
    1.2 Intended Audience and Reading Suggestions
    1.3 Product Scope
    1.4 References
2. Overall Description
    2.1 Product Perspective
    2.2 Product Functions
    2.3 User Classes and Characteristics
    2.4 Operating Environment
    2.5 Design and Implementation Constraints
    2.6 Assumptions and Dependencies
3. External Interface Requirements
    3.1 User Interfaces
    3.2 Software Interfaces
    3.3 Communications Interfaces
4. Analysis Models
    4.1 Use Case Diagram
    4.2 Sequence Diagram
5. System Features
    5.1 Response Input and Validation
    5.2 Claim Extraction
    5.3 Trusted Evidence Retrieval
    5.4 Fact Verification
    5.5 Confidence Scoring
    5.6 Hallucination Detection and Response Processing
    5.7 Evidence and Result Presentation
    5.8 Verification API
    5.9 Trusted Source Management
    5.10 Logging and Review
6. Other Nonfunctional Requirements
    6.1 Performance Requirements
    6.2 Safety Requirements

6.3 Security Requirements
6.4 Software Quality Attributes
6.5 Business Rules
7. Other Requirements
Appendix A: Glossary
Appendix B: Field Layouts
Appendix C: Requirement Traceability Matrix

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the Dehallucination Add-On Module for Enhancing Reliability of AI Applications. The module is intended to reduce incorrect or unsupported statements produced by AI applications by checking factual claims against trusted sources or knowledge graphs before returning a final response.

The SRS covers the proposed software module, its users, interfaces, major processing steps, functional requirements, nonfunctional requirements, assumptions, and traceability. It is written for the Phase-1 mini-project and describes the expected behavior of the system before implementation.

1.2 Intended Audience and Reading Suggestions

This document is intended for the student project team, project guide, developers, testers, reviewers, and other readers who need to understand how the proposed system should work.

- Read Section 1 first for the purpose and scope of the project.
- Section 2 gives the overall architecture, functions, users, environment, and constraints.
- Section 3 describes the interfaces used by the module.
- Section 4 contains the analysis models.
- Section 5 contains the detailed functional requirements.
- Section 6 contains nonfunctional requirements such as performance, security, reliability, and usability.
- The appendices provide terminology, field information, and requirement traceability.

1.3 Product Scope

The proposed product is a software add-on that can be placed between an AI application and its final response delivery path. It accepts AI-generated text, identifies factual claims, searches trusted information sources, compares the claims with the retrieved evidence, and produces a verification result.

The module is not intended to replace the underlying AI model. Its purpose is to provide an additional verification layer that can identify potentially hallucinated content, show supporting evidence, assign confidence information, and return a verified or modified response.

- Accept generated text from a chatbot or content generator.
- Identify factual claims in the generated response.
- Retrieve relevant information from configured trusted sources.
- Compare claims with retrieved evidence.
- Classify claims as supported, contradicted, or uncertain where possible.
- Generate a verification result and confidence information.
- Return a verified or modified response with supporting evidence.
- Maintain logs and verification records for review.

1.4 References

- PES University, Department of Computer Science and Engineering, JACKFRUIT - SE MINIPROJECT GUIDELINES 2026, Software Requirements Specification template.
- Problem Statement 39: Dehallucination Add-On Module for Enhancing Reliability of AI Applications.

Note: Detailed external implementation references may be added during later phases when the exact AI model, retrieval framework, knowledge graph, database, and APIs are finalized.

2. Overall Description

2.1 Product Perspective

The proposed system is a new, self-contained verification add-on that can be integrated with an existing AI application such as a chatbot or content generator. The AI application sends its generated response to the dehallucination module. The module processes the response, retrieves evidence, verifies factual claims, and sends the result back to the application.

The main internal flow is: input processing → claim extraction → evidence retrieval → fact verification → result generation. Trusted external sources provide evidence, while a database can store requests, results, source information, and logs.

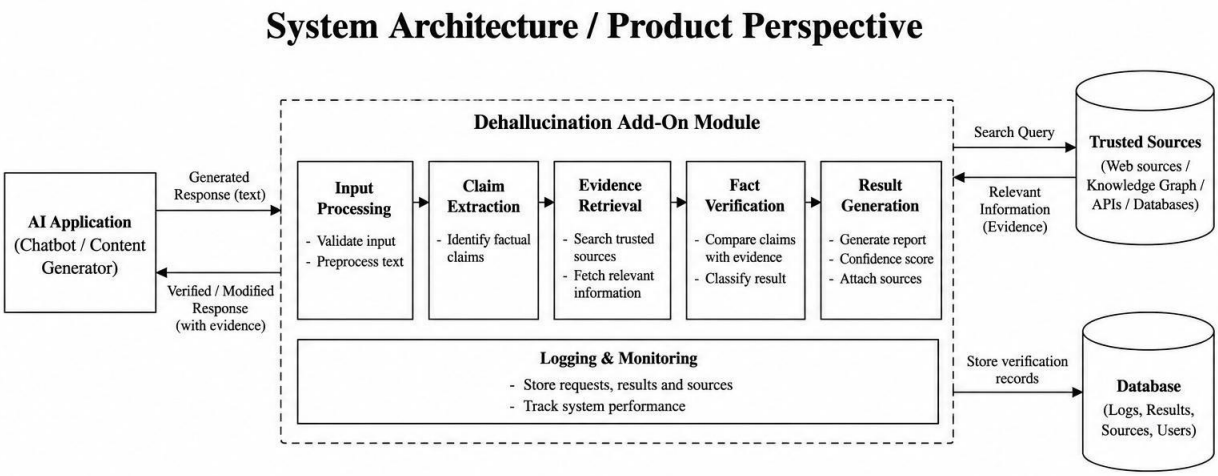


Figure 1: System Architecture of the Dehallucination Module

Figure 1: System Architecture / Product Perspective

2.2 Product Functions

- Receive and validate AI-generated responses.
- Preprocess text before verification.
- Extract factual claims from the response.
- Retrieve relevant evidence from trusted sources or knowledge graphs.
- Compare claims with evidence using a verification engine.
- Calculate or assign a confidence score to verification results.
- Flag unsupported or contradicted claims.
- Generate a verified/modified response and verification report.
- Display supporting evidence and sources.
- Provide an API for integration with other AI applications.
- Manage configured trusted sources.
- Store logs and verification records for review.

2.3 User Classes and Characteristics

- End User – uses the connected chatbot/content generator and views the verified response and evidence. Basic technical knowledge is expected.
- Developer/Integrator – integrates the verification module or API with an AI application. Familiarity with APIs and software integration is expected.
- Administrator – manages trusted sources, configuration, and system logs. Requires higher privileges than a normal user.
- Reviewer – examines flagged or uncertain results and supporting evidence for evaluation and improvement.

2.4 Operating Environment

The system is expected to operate as a software service on a computer or server capable of running the selected programming language, AI/NLP libraries, retrieval components, and database. A webbased interface or API may be used for integration with the target AI application.

- Operating system: Windows or Linux environment suitable for development/deployment.
- Application environment: Python-based or equivalent backend environment.
- Database: relational or document database depending on implementation choice.
- Network: Internet or local network connection when external trusted sources are used.
- Browser: modern web browser for any web-based user interface.
- External services: trusted web sources, APIs, or knowledge graphs as configured.

2.5 Design and Implementation Constraints

- The module must work as an add-on and should not require replacement of the underlying AI model.
- Verification quality depends on the availability and quality of trusted evidence.
- External source rate limits, API availability, and network connectivity may affect retrieval.
- The implementation must clearly separate generated content from retrieved evidence.
- The system should avoid treating unsupported generated text as verified merely because no contradiction was found.

- Authentication and access control are required for administrative functions.
- The exact AI model, database, and source APIs may be finalized during implementation.

2.6 Assumptions and Dependencies

- The connected AI application can provide its generated text to the module.
- At least one trusted source or knowledge source is available for verification.
- The system has network access when external sources are required.
- External APIs or knowledge graphs used by the implementation remain accessible.
- The project team will define source reliability rules during implementation.
- The final confidence-scoring method may depend on the selected verification approach.

3. External Interface Requirements

3.1 User Interfaces

The user interface should be simple and focused on verification. A typical verification screen should allow the user to submit or receive an AI-generated response and then view the verification result.

- Text input or API-based submission of an AI-generated response.
- Verify button or equivalent action.
- Verification status for the response and individual claims.
- Confidence information.
- Supporting evidence and source references.
- Indication of unsupported, contradicted, or uncertain claims.
- Error messages when input or source retrieval fails.
- Administrator screens for source management and logs, if implemented.

3.2 Software Interfaces

The module interfaces with the AI application, claim extraction/NLP components, retrieval components, verification engine, database, and trusted external sources.

- AI application interface: receives generated text and returns a verification result.
- Claim extraction component: receives text and returns factual claims.
- Evidence retrieval component: receives claims/search queries and returns relevant evidence.
- Verification engine: receives claims and evidence and returns verification status and confidence information.
- Database interface: stores and retrieves verification records, source details, and logs.
- Trusted-source interfaces: access web sources, APIs, databases, or knowledge graphs.

3.3 Communications Interfaces

Communication between the AI application and the module may use HTTP/HTTPS APIs. JSON is the expected message format for structured requests and responses. HTTPS should be preferred whenever data is transferred over an external network.

- Request: response text, request identifier, and optional configuration information.
- Response: verification status, modified/verified response, claim results, confidence information, and evidence/source information.
- External retrieval: HTTP/HTTPS or the protocol required by the selected trusted source.
- Communication failures should be reported without presenting incomplete verification as successful verification.

4. Analysis Models

The following analysis models describe the major interactions and processing flow of the proposed system. The models are intentionally kept simple so that the relationship between users, system components, and the verification process is clear.

4.1 Use Case Diagram

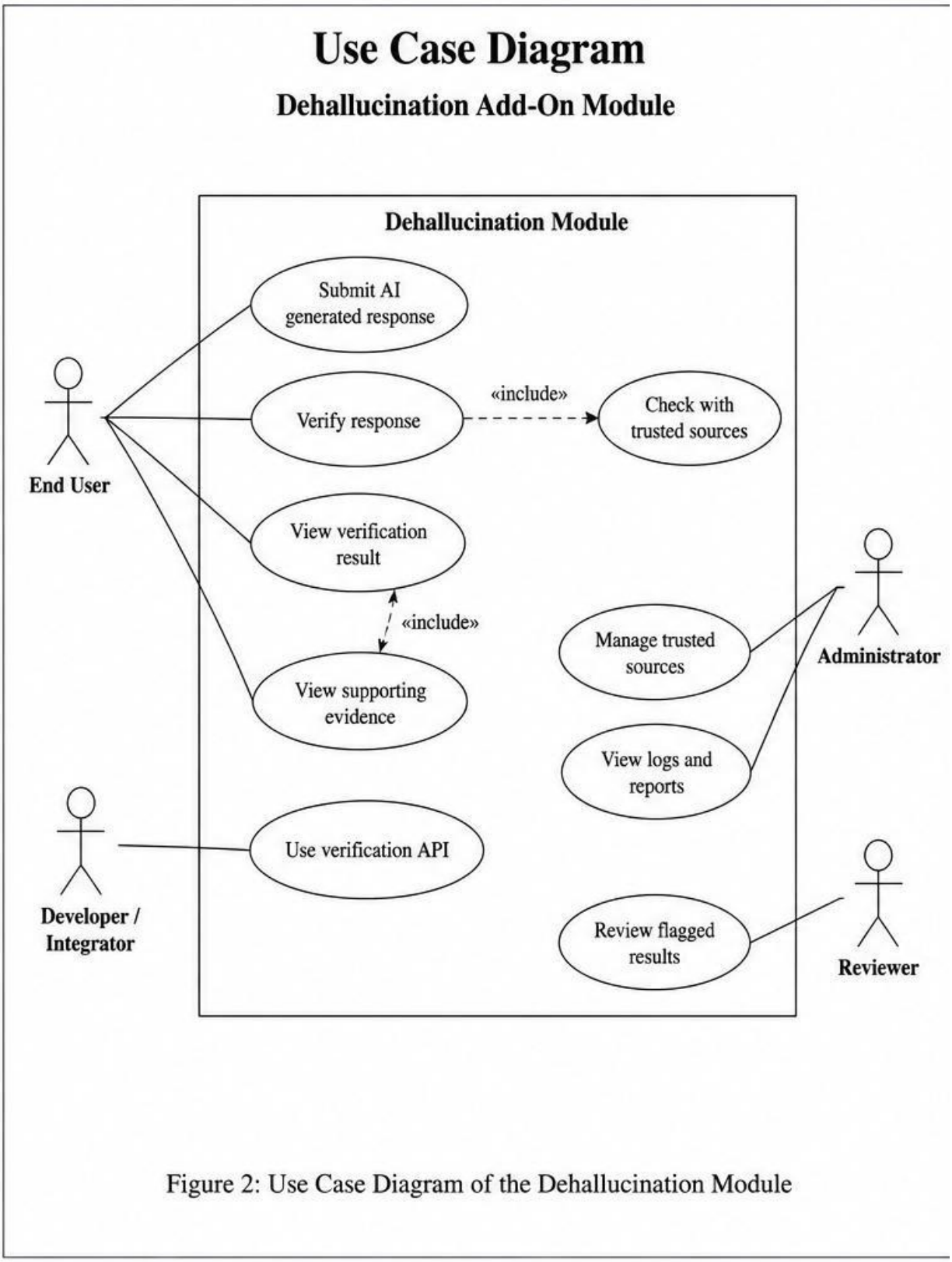


Figure 2: Use Case Diagram of the Dehallucination Module

The main actors are the End User, Developer/Integrator, Administrator, and Reviewer. The End User submits and verifies responses and views results/evidence. The Developer/Integrator uses the verification API. The Administrator manages trusted sources and logs. The Reviewer examines flagged results.

4.2 Sequence Diagram

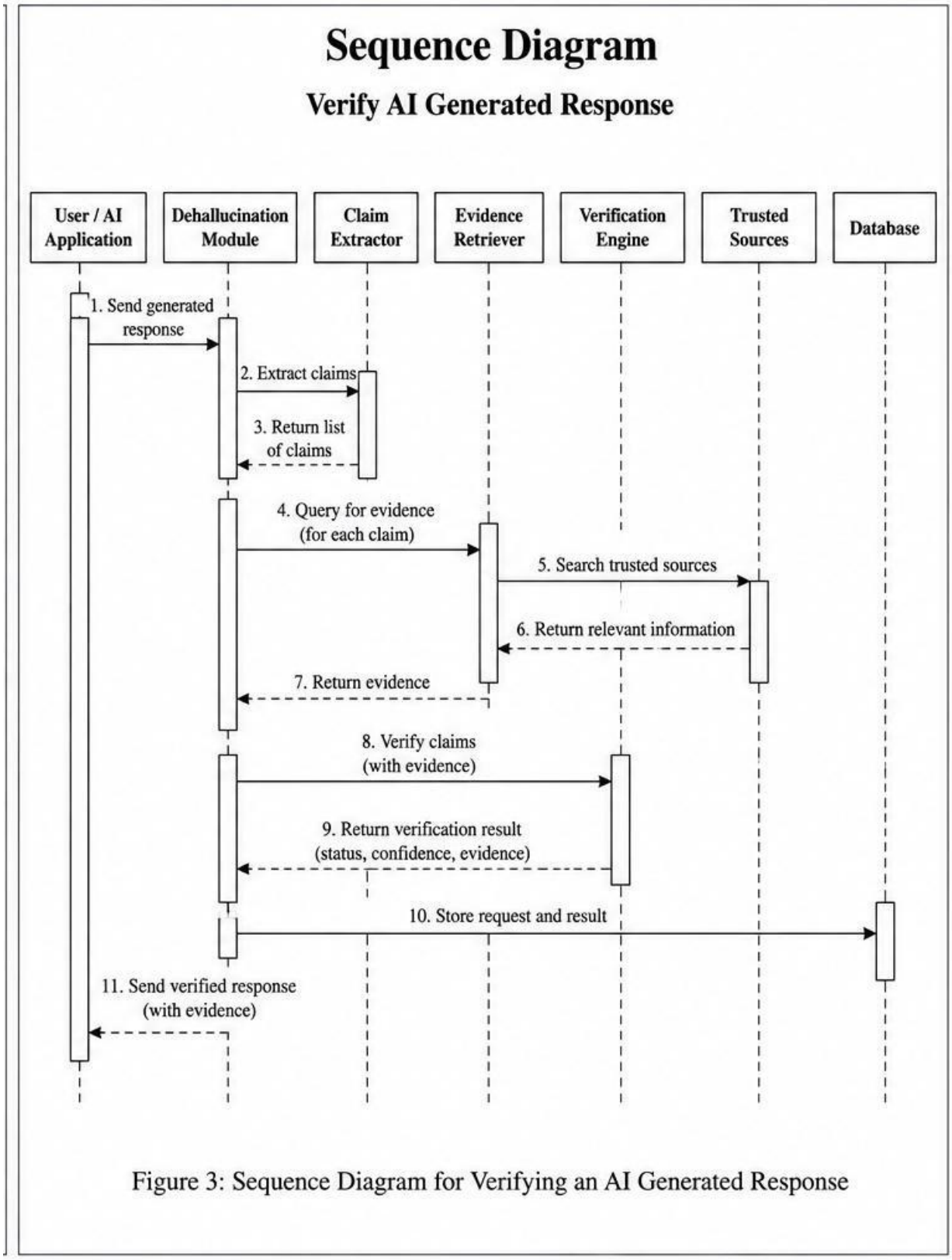


Figure 3: Sequence Diagram for Verifying an AI Generated Response

The sequence begins when the AI application sends generated text to the dehallucination module. Claims are extracted, evidence is requested from trusted sources, and the verification engine compares claims with evidence. The result is stored and returned as a verified response with evidence and verification information.

5. System Features

5.1 Response Input and Validation

Description and Priority: Receives an AI-generated response and checks whether the input is valid before starting verification. Priority: High.

Stimulus/Response Sequences:

1. User/application submits generated response.
2. System validates the request and preprocesses the text.
3. If valid, the system starts claim extraction.
4. If invalid, the system returns an error without performing verification.

Functional Requirements:

FR-01: The system shall accept an AI-generated text response for verification.

FR-02: The system shall validate that the submitted response is present and in the supported format.

FR-03: The system shall reject empty, malformed, or unsupported input and return an appropriate error message.

FR-04: The system shall preprocess valid text before claim extraction.

5.2 Claim Extraction

Description and Priority: Identifies factual statements or claims in the generated response that can be checked against evidence. Priority: High.

Stimulus/Response Sequences:

5. Validated response is passed to the claim extraction component.
6. The component identifies candidate factual claims.
7. The extracted claims are returned for verification.
8. If no verifiable claim is found, the system shall report that result.

Functional Requirements:

FR-05: The system shall identify factual claims from a valid generated response.

FR-06: The system shall maintain the association between each extracted claim and its original response.

FR-07: The system shall handle responses for which no verifiable factual claim is identified.

5.3 Trusted Evidence Retrieval

Description and Priority: Searches configured trusted sources and retrieves information relevant to each claim. Priority: High.

Stimulus/Response Sequences:

9. System creates a search query for a claim.
10. The retrieval component sends the query to trusted sources.
11. Relevant information is returned.
12. Unavailable sources or failed retrievals are recorded.

Functional Requirements:

FR-08: The system shall generate or accept a search query for each claim requiring evidence.

FR-09: The system shall retrieve relevant evidence from configured trusted sources.

FR-10: The system shall record source information associated with retrieved evidence.

FR-11: The system shall report retrieval failure when evidence cannot be obtained.

5.4 Fact Verification

Description and Priority: Compares extracted claims with retrieved evidence and assigns a verification status. Priority: High.

Stimulus/Response Sequences:

13. Claims and evidence are provided to the verification engine.
14. The engine compares the claim with the evidence.
15. Each claim is assigned a status such as supported, contradicted, or uncertain.
16. The result is passed to result generation.

Functional Requirements:

FR-12: The system shall compare each verifiable claim with retrieved evidence.

FR-13: The system shall classify a claim as supported, contradicted, or uncertain according to the implemented verification method.

FR-14: The system shall preserve the evidence used for each verification decision.

FR-15: The system shall not mark a claim as verified when the verification result is uncertain.

5.5 Confidence Scoring

Description and Priority: Provides confidence information for verification results so that users can distinguish strong results from uncertain ones. Priority: Medium.

Stimulus/Response Sequences:

17. Verification result is produced.
18. The scoring component calculates or assigns a confidence value.
19. The confidence information is attached to the claim/result.
20. Low-confidence results may be flagged for review.

Functional Requirements:

FR-16: The system shall assign a confidence score or confidence level to a verification result.

FR-17: The system shall associate confidence information with the corresponding claim.

FR-18: The system shall flag results that fall below the configured confidence threshold, if a threshold is used.

5.6 Hallucination Detection and Response Processing

Description and Priority: Uses claim verification results to identify potentially hallucinated content and decide how the final response should be presented. Priority: High.

Stimulus/Response Sequences:

21. System receives claim-level verification results.
22. Unsupported or contradicted claims are identified.
23. The system decides whether to flag, remove, qualify, or modify affected content according to the configured project rule.
24. A final response is generated.

Functional Requirements:

FR-19: The system shall identify response content associated with contradicted or unsupported claims.

FR-20: The system shall mark potentially hallucinated content for user awareness.

FR-21: The system shall generate a verified or modified response based on the verification results.

FR-22: The system shall retain the original generated response for comparison when logging is enabled.

5.7 Evidence and Result Presentation

Description and Priority: Presents the verification outcome and supporting evidence in a form understandable to the user. Priority: High.

Stimulus/Response Sequences:

25. Verification completes.
26. System prepares claim-level and response-level results.
27. Sources and evidence are attached.
28. The final result is displayed or returned to the requesting application.

Functional Requirements:

FR-23: The system shall display or return the overall verification result.

FR-24: The system shall display or return supporting evidence and source information for verified claims where available.

FR-25: The system shall distinguish supported, contradicted, and uncertain claims in the result.

5.8 Verification API

Description and Priority: Provides an interface through which another AI application can send responses to the module and receive verification results. Priority: High.

Stimulus/Response Sequences:

29. Developer integrates the API.
30. AI application sends a verification request.
31. Module processes the request.
32. API returns the verification result in the defined format.

Functional Requirements:

FR-26: The system shall provide an API endpoint or equivalent interface for response verification.

FR-27: The API shall return a structured verification result containing status and relevant evidence information.

FR-28: The API shall return an appropriate error response for invalid requests or unavailable verification services.

5.9 Trusted Source Management

Description and Priority: Allows authorized administrators to add, remove, enable, disable, or update the trusted sources used by the system. Priority: Medium.

Stimulus/Response Sequences:

33. Administrator authenticates.
34. Administrator views configured sources.
35. Administrator adds or changes source details.
36. System validates and stores the configuration.

Functional Requirements:

FR-29: The system shall allow an authorized administrator to manage the configured trusted sources.

FR-30: The system shall prevent unauthorized users from changing trusted-source configuration.

5.10 Logging and Review

Description and Priority: Maintains records needed to understand verification activity and review flagged results. Priority: Medium.

Stimulus/Response Sequences:

37. System records verification activity.
38. Administrator/reviewer accesses permitted records.
39. Flagged or uncertain results can be reviewed.
40. Logs can be used for debugging and project evaluation.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- The system should begin processing a valid request without unnecessary delay.

- The response time should be acceptable for interactive chatbot use under normal network and source conditions.
- The system should avoid repeated retrieval of identical evidence where caching is implemented.
- Performance should degrade gracefully when an external source is slow or unavailable.
- The exact quantitative response-time target is TBD and should be finalized after the selected implementation and test environment are fixed.

6.2 Safety Requirements

- The system shall not present an uncertain verification result as a confirmed fact.
- When trusted evidence is unavailable, the system should indicate that verification could not be completed.
- The module should preserve uncertainty instead of silently modifying factual content without an indication where practical.
- The system should not be used as the sole safety-critical decision mechanism without domainspecific validation.

6.3 Security Requirements

- Only authorized users shall access administrative functions.
- Trusted-source configuration shall be protected from unauthorized modification.
- API requests should use authentication when exposed beyond a controlled local environment.
- HTTPS should be used for network communication containing protected information.
- Logs should avoid storing unnecessary sensitive user information.
- Stored records should be protected against unauthorized access or modification.

6.4 Software Quality Attributes

- Correctness: verification results should be based on the evidence actually retrieved.
- Reliability: failures in external sources should be handled without falsely reporting successful verification.
- Usability: results and evidence should be easy for a normal user to understand.
- Maintainability: claim extraction, retrieval, and verification components should be separable so they can be updated independently.
- Interoperability: the API should allow integration with different AI applications.
- Testability: each major verification stage should be testable independently.
- Portability: the software should be deployable in commonly used development/server environments.
- Traceability: verification results should be linked to the claims and evidence used to produce them.

6.5 Business Rules

- Only configured trusted sources shall be used as trusted evidence sources.
- Administrative source-management functions shall be restricted to authorized administrators.
- A claim shall not be marked supported without sufficient evidence according to the implemented verification rule.

- Contradicted and uncertain results shall remain distinguishable from supported results.
- Reviewer access shall be limited to the records required for evaluation and review.

7. Other Requirements

- Database requirements: the implementation should store verification requests, claim results, evidence/source information, and logs when persistence is enabled.
- Data format requirement: API requests and responses should use a consistent structured format such as JSON.
- Configuration requirement: trusted sources, confidence thresholds, and other configurable parameters should be maintained separately from core verification logic.
- Documentation requirement: the project should document setup, API usage, source configuration, and known limitations.
- Future extensibility: additional trusted sources, knowledge graphs, verification methods, and AI applications should be addable without redesigning the complete system.

Appendix A: Glossary

AI: Artificial Intelligence.

API: Application Programming Interface used for communication between software components.

Claim: A factual statement extracted from AI-generated text that can be checked against evidence.

Dehallucination: The process of detecting and reducing unsupported or incorrect statements produced by an AI system.

Evidence: Information retrieved from a trusted source and used to verify a claim.

Fact Verification: Comparison of a factual claim with available evidence to determine its support.

Hallucination: AI-generated information that is unsupported, incorrect, or presented as factual without adequate basis.

Knowledge Graph: A structured representation of entities and relationships that can be used as a source of factual information.

Confidence Score: A value or level indicating how strongly the system supports a verification result.

Trusted Source: A configured information source considered suitable for evidence retrieval.

Verification Result: The status and supporting information produced after comparing a claim with evidence.

Reviewer: An authorized user who examines flagged or uncertain verification results.

Appendix B: Field Layouts

The following fields represent the main data items expected in the verification request/result and can be used as the basis for the required field-layout spreadsheet in the project.

Field	Data Type	Length	Mandatory	Description

Request ID	String	TBD	Yes	Unique identifier for a verification request.
Generated Response	Text	TBD	Yes	AI-generated text submitted for verification.
Claim ID	String	TBD	No	Identifier for an extracted claim.
Claim Text	Text	TBD	No	Factual claim extracted from the response.
Verification Status	String	30	Yes	Supported, Contradicted, or Uncertain.
Confidence	Numeric	TBD	No	Confidence value or level assigned to the result.
Evidence	Text	TBD	No	Relevant information used to verify the claim.
Source	String/URL	TBD	No	Trusted source associated with the evidence.
Timestamp	Date/Time	TBD	Yes	Time at which the verification was performed.
User/Client ID	String	TBD	No	Identifier of the requesting user/application where applicable.
Error Code	String	TBD	No	Code describing a validation or retrieval failure.

Report requirements: A verification report should contain the request ID, original response, extracted claims, verification status, confidence information, supporting evidence, sources, timestamp, and any errors or warnings.

Appendix C: Requirement Traceability Matrix

The traceability matrix links the major functional requirements to the system features and the corresponding analysis/use-case areas. It can be expanded during implementation and testing.

Requirement Range	**System Feature**	**Related Use Case / Model**

FR-01 – FR-04	Response Input and Validation	Submit AI generated response
FR-05 – FR-07	Claim Extraction	Verify response / Sequence Diagram
FR-08 – FR-11	Trusted Evidence Retrieval	Check with trusted sources / Sequence Diagram
FR-12 – FR-15	Fact Verification	Verify response / Sequence Diagram
FR-16 – FR-18	Confidence Scoring	View verification result
FR-19 – FR-22	Hallucination Detection and Response Processing	Verify response / View result
FR-23 – FR-25	Evidence and Result Presentation	View verification result / View supporting evidence
FR-26 – FR-28	Verification API	Use verification API
FR-29 – FR-30	Trusted Source Management	Manage trusted sources

Note: Detailed test-case traceability will be added during the implementation/testing phase. The PES guideline specifies that later testing should cover the implemented use cases with test cases.